

SEA SENTRY (PHASE-3)

(FISHERMAN BORDER ALERT SYSTEM)

TEAM MEMBERS:

AASHIKA M

INIYA M

DEEPIKA G

SYSTEM ARCHITECTURE:

OVERALL SYSTEM ARCHITECTURE:

1. NEO-6M GPS Module:

2. The NEO-6M GPS module is used to obtain the geographical position of the fisherman. It provides important location information such as **latitude and longitude**.

3. Arduino Mega 2560:

The Arduino Mega 2560 acts as the **central processing unit** of the system. It receives the location data and processes it to determine the fisherman's position with respect to the predefined maritime boundary.

4. Boundary Monitoring:

The obtained GPS coordinates are compared with the **predefined safe-zone and boundary coordinates**. This enables the system to identify whether the fisherman is within the safe region or approaching/crossing the boundary.

5. Warning Generation:

When the fisherman approaches a critical boundary region, the Arduino generates a warning signal. This provides an opportunity for the fisherman to take corrective action.

6. 3-in-1 LED Indicator:

The three-color LED provides a **visual indication of different system conditions**. Different colors can represent safe, warning, and danger/boundary-crossing conditions.

7. Buzzer Alert:

The buzzer provides an **audible warning** when the fisherman reaches a warning or danger condition. Different buzzer patterns can be used for different levels of alerts.

8. PROTEUS Simulation:

PROTEUS is used to simulate and test the system before implementing the physical prototype. It helps verify the Arduino program, circuit connections, input conditions, and alert responses virtually.

9. Real-Time Monitoring:

The system is designed to continuously process the location information so that changes in the fisherman's position can be monitored during operation.

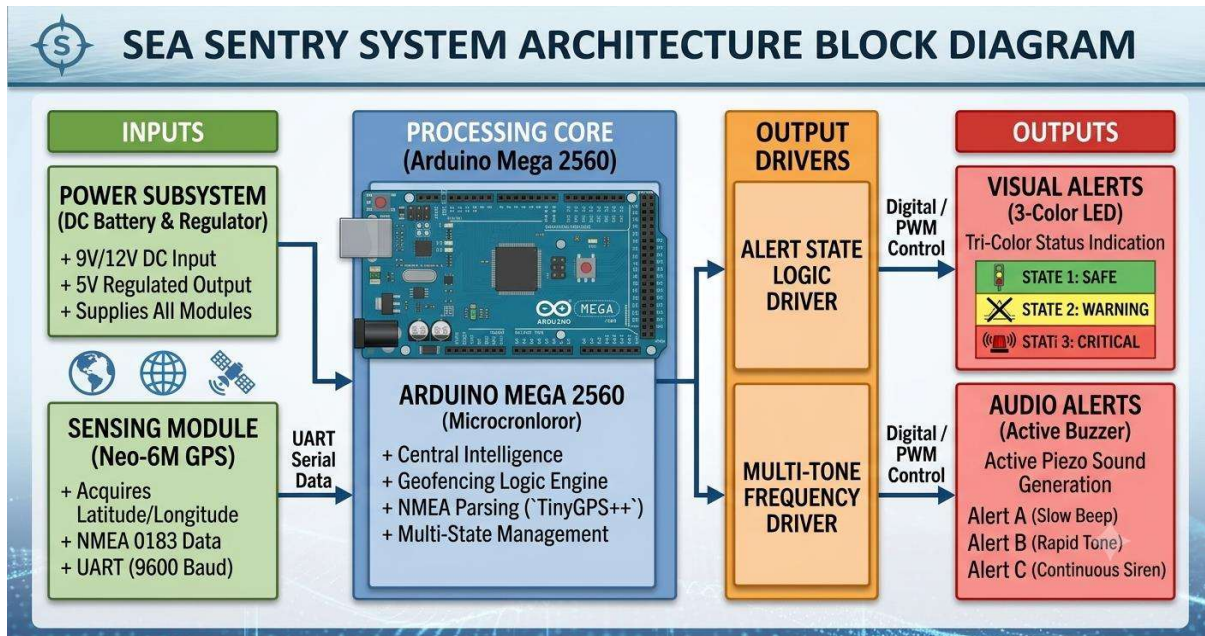
10. Early Warning:

The main purpose of the architecture is to provide an **early warning before the fisherman crosses the predefined boundary**, helping improve awareness and safety.

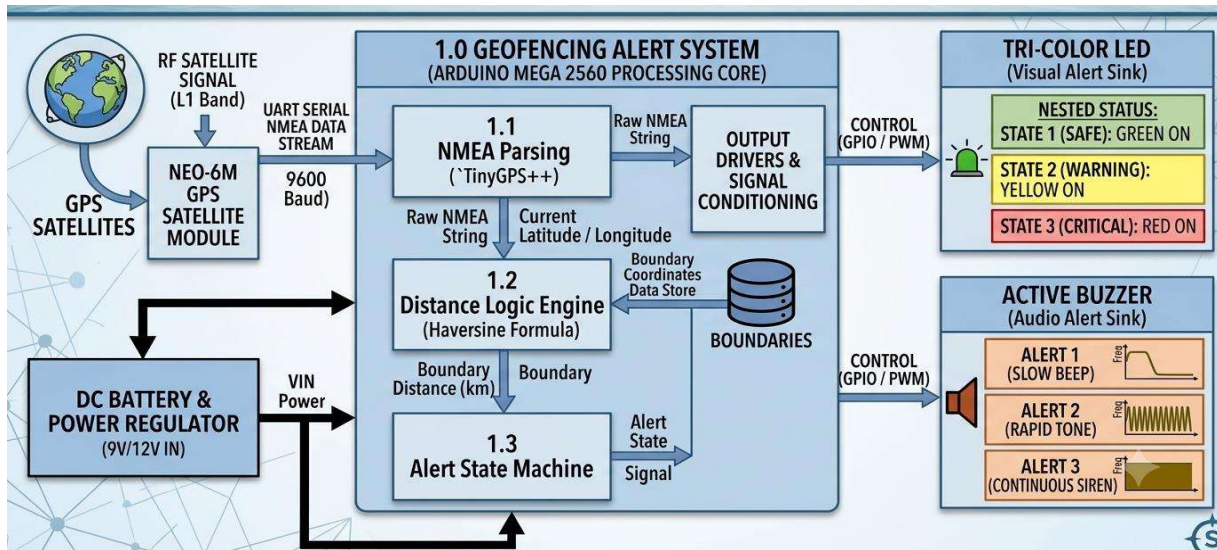
11. Overall Working:

Thus, the system follows the basic flow:

GPS Location → Arduino Processing → Boundary Checking → Warning Decision → LED + Buzzer Alert.



DATA FLOW DIAGRAM:



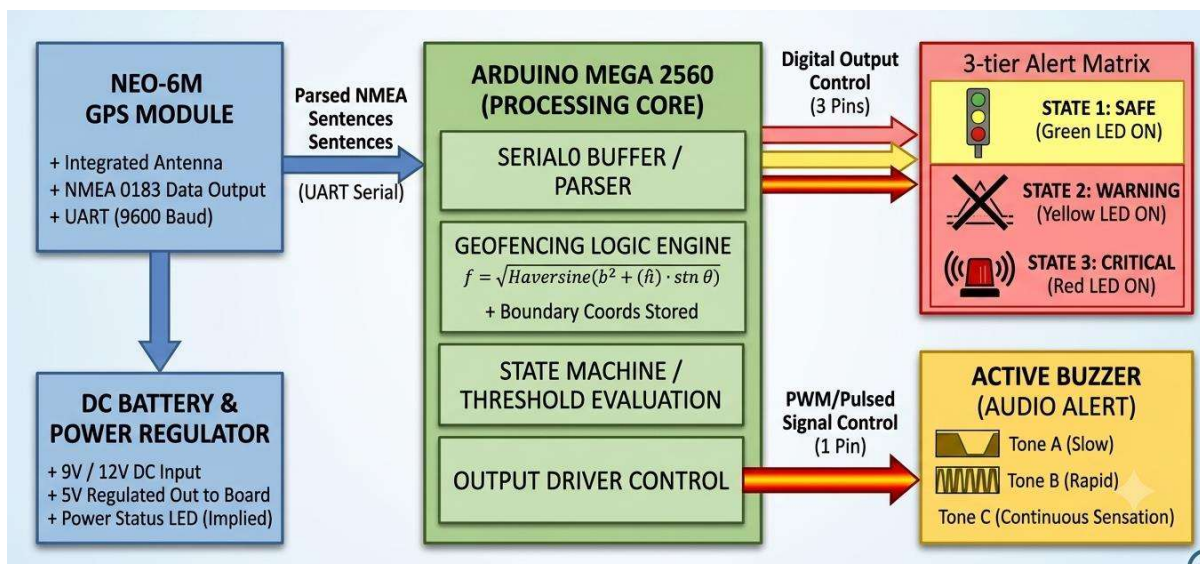
FUNCTIONAL BLOCK DIAGRAM:

The **functional block diagram** of the Sea Sentry system represents the major functions performed by each component and the flow of information between them. The system begins with the **NEO-6M GPS module**, which receives signals from GPS satellites and provides the current latitude and longitude of the fisherman.

The GPS coordinates are then given to the **Arduino Mega 2560**, which acts as the main processing unit. The Arduino processes the received location information and compares it with the **predefined maritime boundary coordinates**. Based on this comparison, the system determines the current status of the fisherman, such as **safe, warning, or danger**.

According to the detected status, the Arduino controls the **3-in-1 LED indicator** to provide a visual indication. At the same time, the **buzzer** generates an audible alert, with different alert patterns representing different warning conditions.

The complete functionality is **simulated and tested using PROTEUS**, which allows the circuit operation and program logic to be verified before implementing the physical prototype.



HARDWARE SOFTWARE PARTITIONING:

Hardware part:

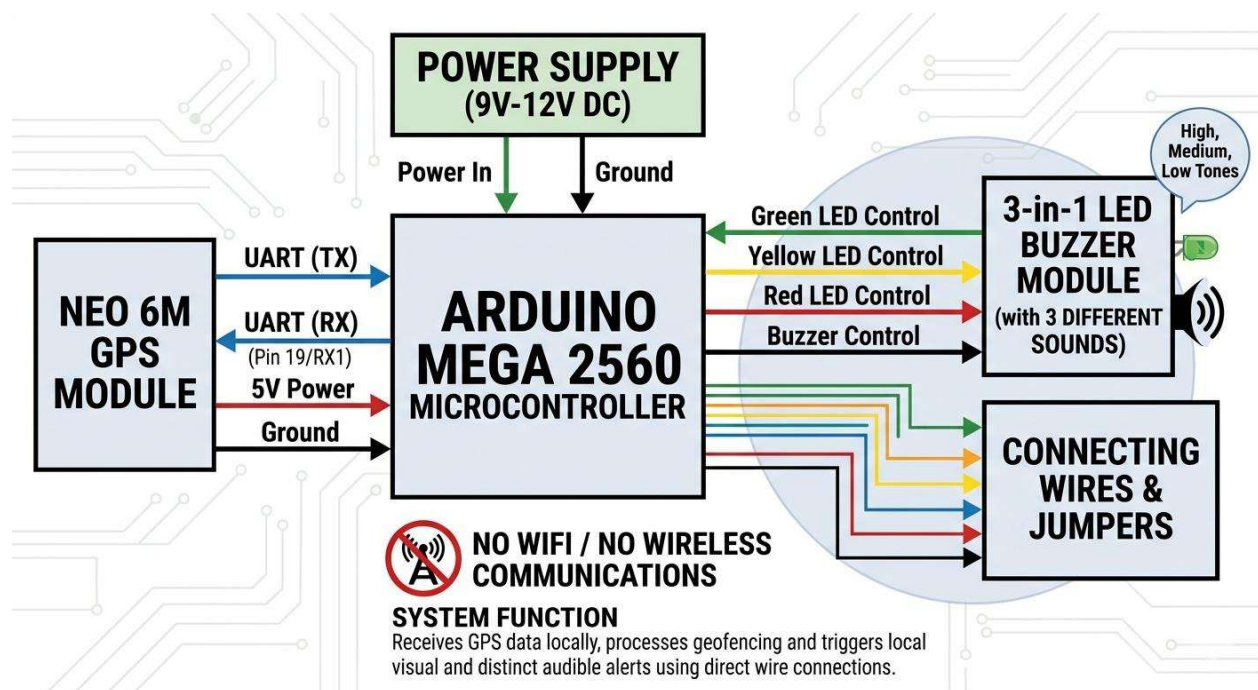
- **Arduino Mega 2560** – main processing/controller hardware
- **NEO-6M GPS module** – obtains location coordinates
- **3-in-1 LED** – visual warning indicator
- **Buzzer** – audible warning
- Connecting wires and power supply – provide electrical connections and power

Software part:

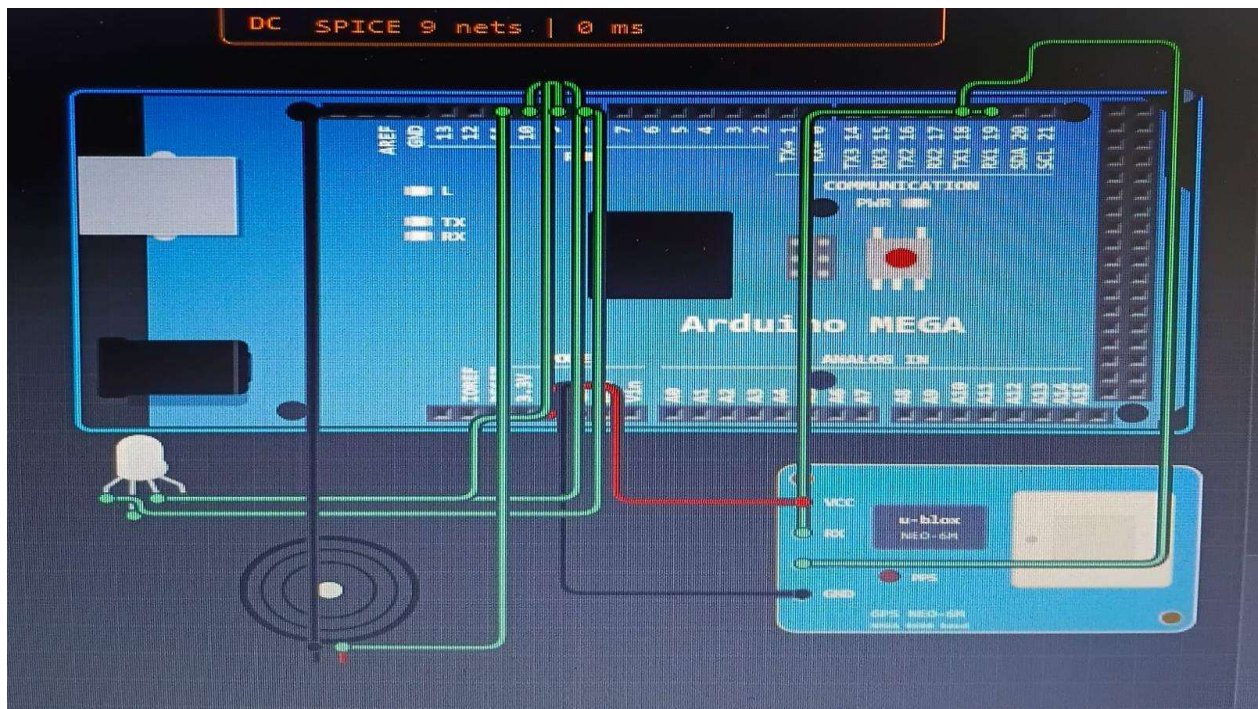
- The **Arduino program** contains the logic for reading GPS data.
- It processes the **latitude and longitude** values.
- It compares the current position with the **predefined boundary coordinates**.
- It determines the system status, such as **safe, warning, or danger**.
- It controls the LED and buzzer according to the detected status.
- **PROTEUS** is used to simulate and test the hardware and software

HARDWARE DESIGN:

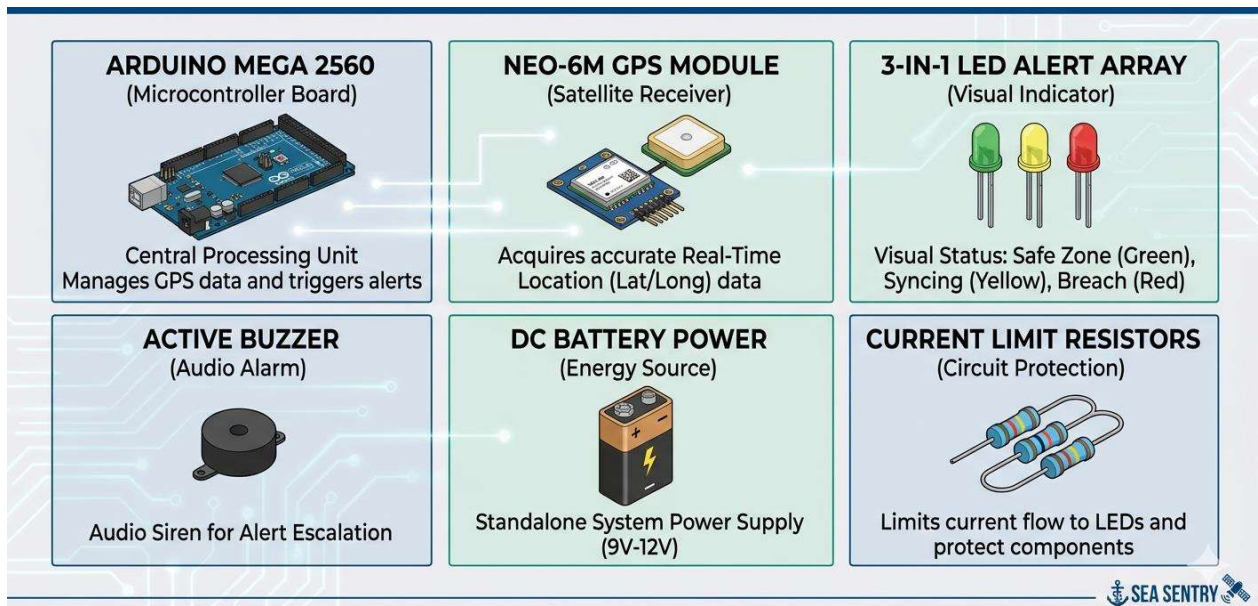
FINAL BLOCK DIAGRAM:



CIRCUIT SCHEMATICS:



COMPONENT SELECTION:



1. Processing Core: Arduino Mega 2560

- **Multi-UART Architecture:** Features **4 independent hardware serial ports**. This allows dedicated communication with the GPS module on `Serial1[1]` without interrupting debugging or locking up the processor loop.
- **Memory Buffer:** Packed with **8 KB of SRAM**, providing ample memory to process and parse long incoming GPS NMEA text strings without crashing.

2. Telemetry Unit: Neo-6M GPS Module

- **Autonomous Operation:** Tracks global satellite coordinates independently without relying on Wi-Fi or cellular networks.
- **Standardized Output:** Converts complex satellite signals into standard **NMEA-0183 ASCII data sentences**, transmitting clean location updates to the Arduino Mega using minimal power (approx. 45mA).

3. Alert Interface: 3-Tier LED Array & Active Buzzer

- **Visual Recognition:** Uses distinct high-efficiency **Green, Yellow, and Red LEDs** for instant status awareness (Safe, Searching, or Geofence Breach) under open daylight conditions.
- **Acoustic Penetration:** Uses a single **Active Buzzer** driven at three distinct pulse frequencies to pierce through outdoor marine noise (wind, waves, engines) matching the risk levels.

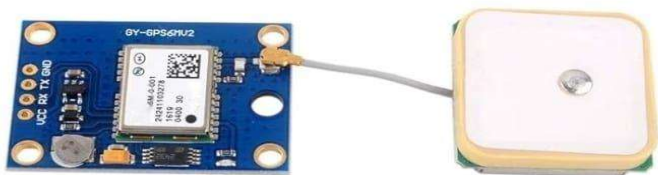
4. System Protection: 220 Ω Resistors & DC Battery

- **Current Regulation:** Incorporates **220 Ω inline resistors** to limit current draw to roughly 13.6mA, protecting the Arduino pins and LEDs from electrical burnout.
- **Regulated Power:** A standalone **9V–12V DC Battery** offers mobile, stable power, relying on the Arduino's onboard voltage regulator to down-step it cleanly to a safe 5V rail.

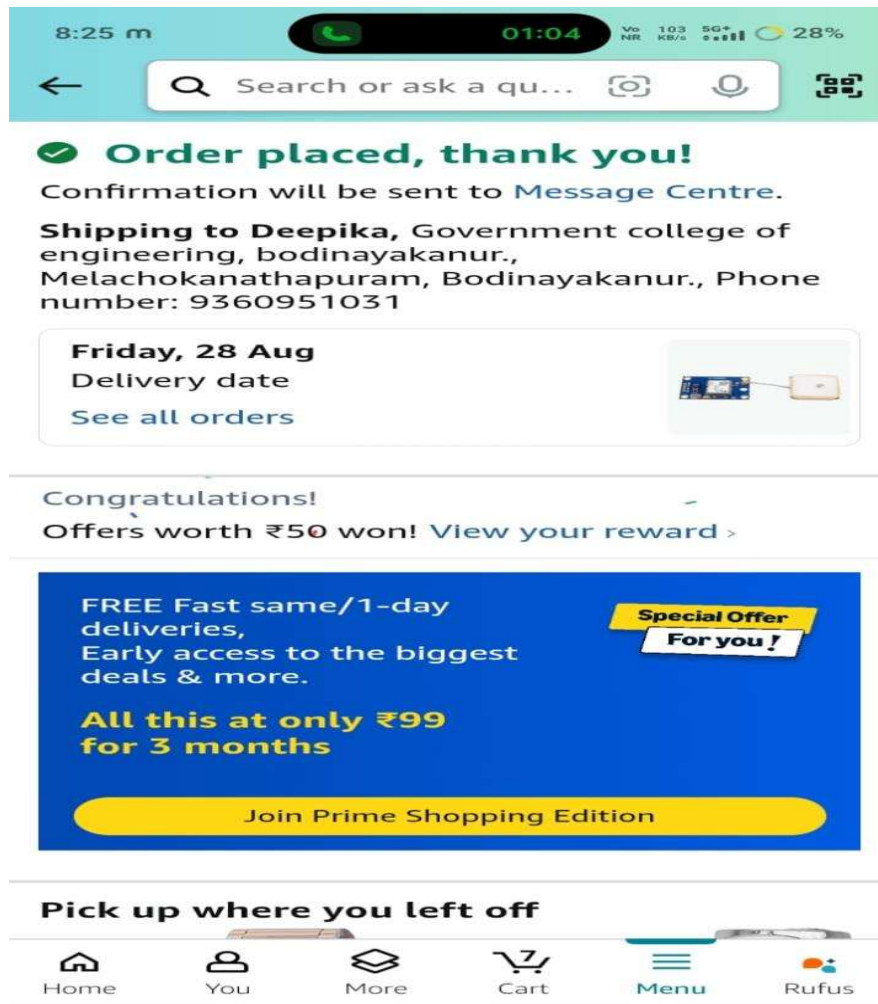
BILL OF MATERIALS (BOM):

Brand: Generic 1.0 ★★★★★ (1)
NEO-6M GPS Module with EEPROM Built-in and Active
GPS Antenna

23%
off

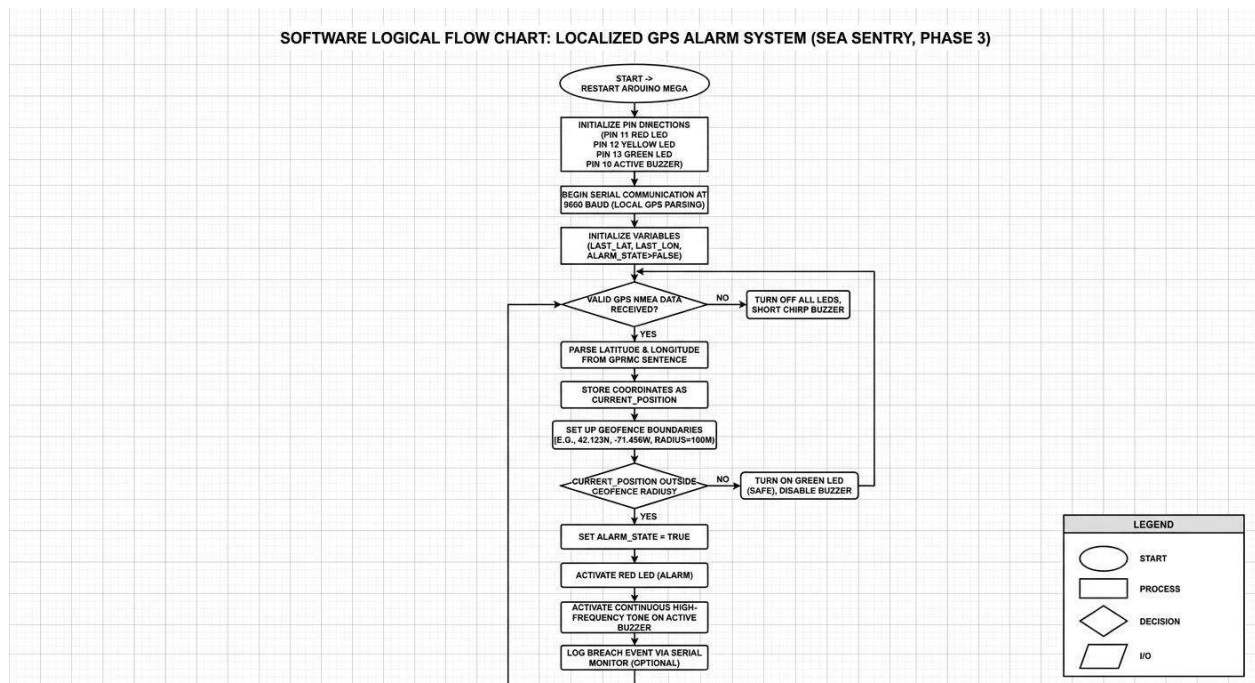


-23% ₹ **385** (₹38,500 /100 g)
[Price history](#)



SOFTWARE DESIGN:

FLOWCHARTS:



ALGORITHMS:

- **Step 1:** Define the target safe coordinates (SAFE_LAT, SAFE_LON) and the maximum allowable boundary radius (SAFE_RADIUS_METERS).
- **Step 2:** Configure the data direction registers by setting Pin 13 (**Green LED**), Pin 12 (**Yellow LED**), and Pin 11 (**Red LED**) as OUTPUT.
- **Step 3:** Set Pin 10 (**Active Buzzer**) as OUTPUT to drive the audio alerts.
- **Step 4:** Open the primary debugging hardware port (Serial0) at **9600 baud** to stream text directly to your Proteus Virtual Terminal window.
- **Step 5:** Open the auxiliary serial hardware port (Serial1) at **9600 baud** to establish a direct UART data line with the **Neo-6M GPS Module**.
- **Step 6:** Read the incoming data buffer on Serial1 to check for active **NMEA-0183** sentences from satellites.
- **Step 7: IF** there is no valid data stream or satellite lock, turn on the **Yellow LED** (Pin 12), pulse a slow warning chirp on the **Buzzer** (Pin 10), and loop back to **Step 6**.
- **Step 8: ELSE** (when satellite lock is successful), extract the current latitude and longitude variables from the incoming \$GPRMC text string.
- **Step 9:** Calculate the real-time distance between your current coordinates and the safe target coordinates using the **Haversine mathematical equation**.
- **Step 10: IF** the calculated distance is less than or equal to SAFE_RADIUS_METERS, turn on the **Green LED** (Pin 13) and turn off all other LEDs and the buzzer.

- **Step 11: ELSE** (if the vessel drifts outside the boundary), turn on the **Red LED** (Pin 11) and drive a continuous, high-frequency emergency siren on the **Buzzer** (Pin 10).
- **Step 12:** Print the live distance and alarm status variables to the Proteus Virtual Terminal, delay the system for 500 milliseconds, and loop back to **Step 6**.

COMMUNICATION PROTOCOLS:

- **Protocol Type:** Universal Asynchronous Receiver-Transmitter (UART) / Serial Communication.
- **Data Layout Configuration:** 8 Data Bits, No Parity, 1 Stop Bit (Standard **8N1** framing).
- **Baud Rate Speed:** Fixed at **9600 bps** (bits per second) to match the native hardware output of the Neo-6M GPS module.
- **Physical Interconnect Layer:** Standard TTL logic levels (0V for Logic LOW, 5V for Logic HIGH).

2. Dual-Channel Protocol Architecture

The **Arduino Mega 2560** acts as the communication hub, running two separate hardware UART channels simultaneously without data collisions:

Channel 1: Telemetry Ingest (Serial1)

- **Source Device:** Neo-6M GPS Module (Pin: TX)
- **Destination Device:** Arduino Mega 2560 (Pin 19: RX1)
- **Data Flow:** Unidirectional (Receive Only). The GPS continuously pushes geolocation data packs into the Mega's serial hardware ring buffer.

Channel 0: Diagnostic Telemetry (Serial0)

- **Source Device:** Arduino Mega 2560 (Pin 1: TX0)
- **Destination Device:** Proteus Virtual Terminal Instrument (Pin: RXD)
- **Data Flow:** Unidirectional (Transmit Only). The Mega streams human-readable distance status metrics directly to the simulation screen monitor.

3. Application Data Layer (NMEA-0183 Payload)

The data payload passing over the UART wire follows the global **NMEA-0183** marine electronics text standard. The system captures and processes the **\$GPRMC** (Recommended Minimum Specific GNSS Data) sentence packet frame structure:

- **String Frame Synchronization:** The hardware looks for the ASCII \$ symbol to trigger the parsing loop.
- **Validation Payload Field:** The third array block indicates lock quality (A = Valid Satellite Data, V = Void Data/Searching).
- **Payload Extraction Matrix:** When the state reads A, the software parses the target latitude and longitude fields directly out of the comma-separated ASCII sequence to calculate your geofence boundary boundaries.

SIMULATION:

